

3.2.3 `FrameLayout`

`FrameLayout` places its children like a deck of cards. The latest child covers the previous. This is useful for tabs on a layout; for instance, it's also used as a placeholder for other views, which can be added programmatically at a later time.

3.2.4 `RelativeLayout`

`RelativeLayout`, as the name suggests, lays out its children relative to each other. This position is based on the ID set for the particular child view. `RelativeLayout` doesn't require nesting of layouts to create a certain look, and minimizes the total number of widgets that need to be drawn. This improves the overall performance of the application. `RelativeLayout` positions a child relative to other children based on the ID set for the particular child view.

3.2.5 `AbsoluteLayout`

You can define absolute coordinates on the home screen to define the position of children using the `AbsoluteLayout`. It's a layout for WYSIWYG tools. However, the simplicity of its implementation and attractiveness come at the cost of flexibility. It doesn't adapt well to changes in size or orientation of the screen.

3.3 Getting Started

Here we'll start our project. Open Eclipse and click on File > New > Android Project.

- **Project Name:** Eclipse organizes your project by the name you provide here. Try not to use any spaces for easy access via command line at a later stage.
- **Contents:** Set to 'Create new project in workspace'.
- **Build Target:** Indicate the type of Android system you intend to run the application on.
- **Application name:** A text name for your app
- **Package name:** Adhere to Java package naming conventions for this:
 - Defined using hierarchical naming pattern
 - Levels in hierarchy separated by periods (.)
 - Packages lower in hierarchy are sub-packages of the immediate higher package
 - Naming convention avoids the possibility of two published packages with the same name. This allows for unique namespaces for packages widely distributed.